

SEP Virtual Platform — Build Fix Report

Changes Required to Compile and Link sep-vp on macOS ARM64 with SystemC 3.0.2 / C++20

Tenstorrent Platform Engineering

June 2026

Contents

Executive Summary	2
1 C++20 Language Compliance Issues	3
1.1 Ambiguous form_report_string overloads — csml_report.h	3
1.2 Ambiguous sc_core::sc_time constructor — och_sep_ss.hpp	4
1.3 -Werror, -Wstack-exhausted on deeply-recursive template — csml_register.h ...	5
2 Build-System / CMake Wiring Issues	5
2.1 Broken SoftFloat sub-build — VeeR-ISS/softfloat/CMakeLists.txt	5
2.2 Hard-coded bare -lboost_* linker flags — platform/sep/CMakeLists.txt	6
3 macOS / Apple Platform-Specific Issues	7
3.1 x86 SSE intrinsics included on ARM64 — VeeR-ISS/float.cpp	7
3.2 OVERFLOW / UNDERFLOW macro collision from macOS <math.h> — spi_controller_register.h	7
3.3 stdc++fs library does not exist on macOS — sep/cpu/CMakeLists.txt	8
4 SystemC 3.x API Deprecation	8
4.1 SC_HAS_PROCESS deprecated in IEEE 1666-2023	8
5 Summary Table of All Changes	10
6 Runtime & Test-Infrastructure Errors (run_all_peripherals.sh)	12
6.1 All Debug / ASAN builds failing — C++17 / C++20 ABI mismatch	12
6.2 csrng Debug / ASAN build failing — SystemC::systemc target not found at configure time	12
6.3 Coverage build failing — grep: invalid option -- P	13
6.4 Coverage build failing — FindThreads failure under --coverage flags (macOS) ..	14
6.5 Coverage build failing — ld: library 'gcov' not found (macOS)	14
6.6 Coverage report generation failing — lcov not installed; LLVM gcov format errors	15
6.7 Final peripheral test results after all fixes	16
7 Functional Defects Root-Caused & Fixed (for Vayavya review)	17
7.1 kmac — SHAKE/cSHAKE extended-output and cSHAKE reference defects	17

Defect 1 — Test reference helpers call EVP_DigestFinalXOF per block (TC-031, TC-032, TC-044, TC-182)	17
Defect 2 — Model RUN command calls EVP_DigestFinalXOF a second time (TC-034)	18
Defect 3 — TC-094 reference computes plain SHAKE instead of cSHAKE	18
7.2 edn — unbalanced LCOV_EXCL_START blocks coverage report	18
8 Recommendations	19
Appendix A — Source Diffs for Functional Defects (§7)	20
A.1 kmac_func002_test.cpp & kmac_func003_test.cpp — XOF reference helpers	20
kmac_func002_test.cpp — compute_shake128_reference_blocks and	
compute_shake256_reference_blocks	20
kmac_func003_test.cpp — compute_cshake_reference_blocks	21
A.2 kmac.cpp & kmac.h — Model SHAKE/cSHAKE PROCESS and RUN	
commands	21
A.2a kmac/include/kmac.h — enlarge xof_full_output	22
A.2b kmac/src/kmac.cpp — SHAKE/cSHAKE PROCESS branch	22
A.2c kmac/src/kmac.cpp — SHAKE/cSHAKE RUN branch (the else block under	
kmac_en && xof_total_length > 0)	23
A.3 kmac_func008_test.cpp — TC-094 cSHAKE128 reference and message length .24	
A.3a The reference function	24
A.3b TC-094 call site — message length fix	25
A.4 edn/src/edn.cpp — missing LCOV_EXCL_STOP in handle_write_ALERT_TEST	25
Appendix B — Source Diffs for CSML C++20 Patches (§1.1 and §1.3)	26
B.1 sep/utils/csml/inc/csml_report.h — Replace ambiguous multi-overload set	
(§1.1)	27
B.2 sep/utils/csml/inc/csml_register.h — Clang -Wstack-exhausted pragma guard	
(§1.3)	28

Executive Summary

This report documents all source-code, build-system, and test-infrastructure changes needed to successfully compile, link, and run the SEP Virtual Platform (sep-vp) and all 17 SEP peripheral model test suites against **SystemC 3.0.2**, **Boost 1.90**, **OpenSSL 3**, and **C++20** on **macOS 15 (Apple Silicon / ARM64)**.

The fixes fall into six categories:

Category	Count	Portable to Linux?
C++20 language standard compliance	3	Yes — will fail on any C++20 build
Build-system / CMake wiring	5	Yes — broken on every platform
macOS / Apple Clang platform specifics	3	No — Linux-only workarounds provided
SystemC 3.x API deprecation	1	Yes — deprecated across all platforms
Test infrastructure (run_all_peripherals.sh)	6	Mix — see Section 6
Functional model & test correctness (kmac, edn)	4	Yes — defects in model/test/source on all platforms

Note for the Vayavya team: Section 7 and **Appendix A** document the functional defects that were root-caused and fixed in the kmac model, the kmac test suite, and the edn source. These are genuine correctness issues independent of the porting effort and warrant review. Appendix A contains the exact source-level diffs.

Appendix B contains the exact source-level diffs for the two CSML submodule patches (§1.1 csml_report.h and §1.3 csml_register.h). Both are required for C++20 conformance and are pending upstream merge to Vayavya-Labs/CSML; until merged, CI applies them automatically via a post-checkout patch script.

1 C++20 Language Compliance Issues

These defects exist in the source code itself and will fail to compile with **any** C++20-conformant compiler on **any** operating system (GCC 10+, Clang 13+, MSVC 2019+).

1.1 Ambiguous form_report_string overloads — csml_report.h

File: sep/utils/csml/inc/csml_report.h

Error:

```
error: call to 'form_report_string' is ambiguous
  CSML_REPORT(WARNING, "READ_ERROR", this->register_name, " is write only
  register");
```

Root Cause: C++20 tightened partial ordering rules for function templates. The original code contained two competing variadic overloads:

```
// Overload A – generic head
template<class T, class... va_args>
inline std::string form_report_string(T arg1, va_args... args);

// Overload B – std::string head (more specific in C++17; now ambiguous in C++20)
template<class... va_args>
inline std::string form_report_string(std::string arg1, va_args... args);
```

When called as `form_report_string(std::string, const char*)`, C++20 can no longer determine that Overload B is strictly more specialized than Overload A, so both are equally viable and the call is rejected as ambiguous.

Fix: Replace both overloads with a single, unambiguous two-or-more-argument template that requires at least two parameters (First + Second), eliminating any competition with the single-argument overloads:

```
template<class First, class Second, class... Rest>
inline std::string form_report_string(First first, Second second, Rest... rest)
{
    return form_report_string(first) + form_report_string(second, rest...);
}
```

Affected subsystems: All peripherals that include `csml_register.h` → `csml.h` → `csml_report.h` (virtually the entire SEP peripheral library: `csrng`, `hmac`, `otbn`, `spi_controller`, `uart_16550`, `gpio`, `aes`, `kmac`, ...).

Portability: This fix is required on **all** platforms when compiling with C++20.

1.2 Ambiguous `sc_core::sc_time` constructor — `och_sep_ss.hpp`

File: `vp/platform/sep/och_sep_ss.hpp`

Error:

```
error: call to constructor of 'sc_core::sc_time' is ambiguous
    sc_core::sc_time(globalQuantumNs.get_param_value(), sc_core::SC_NS)
```

Root Cause: `get_param_value()` returns a CCI parameter type that is implicitly convertible to both `double` and `sc_dt::uint64`. SystemC 3.0.x exposes both `sc_time(double, sc_time_unit)` and `sc_time(uint64, sc_time_unit)` constructors. In C++17 the compiler picked `double`; in C++20 neither implicit conversion is preferred over the other.

Fix: Add an explicit `static_cast<double>`:

```
tlm::tlm_global_quantum::instance().set(
    sc_core::sc_time(static_cast<double>(globalQuantumNs.get_param_value()),
    sc_core::SC_NS));
```

Portability: Required on **all** platforms with C++20.

1.3 -Werror, -Wstack-exhausted on deeply-recursive template —

csml_register.h

File: sep/utils/csml/inc/csml_register.h

Error:

error: stack nearly exhausted; compilation time may suffer,
and crashes due to stack overflow are likely [-Werror,-Wstack-exhausted]
csml_reg_vector<otbn::IMEM_type<32>, 4096> IMEM;

Root Cause: csml_reg_vector<T, N> is a recursive template structure. The OTBN peripheral instantiates it with N = 4096, generating a 4096-deep template recursion chain. Clang's -Wstack-exhausted diagnostic fires as a warning, which is then promoted to an error by the project-wide -Werror flag.

Fix: Suppress the diagnostic around the csml_reg_vector class definition using Clang pragmas (GCC does not emit this diagnostic):

```
#if defined(__clang__)
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wstack-exhausted"
#endif

template <class T, unsigned int Quantity>
class csml_reg_vector { /* ... */ };

#if defined(__clang__)
#pragma clang diagnostic pop
#endif
```

Portability: This is a Clang-only diagnostic (-Wstack-exhausted does not exist in GCC). The #if defined(__clang__) guards ensure the fix is a no-op on Linux GCC builds. However, **the deep template recursion is a latent performance issue on all compilers** and should be refactored to an iterative (std::array-based) design for long-term health.

2 Build-System / CMake Wiring Issues

These are missing or incorrect CMake dependency declarations that cause compilation or link failures on **every** platform.

2.1 Broken SoftFloat sub-build — VeeR-ISS/softfloat/CMakeLists.txt

File: sep/cpu/VeeR-ISS/softfloat/CMakeLists.txt

Error:

No targets specified and no makefile found.

Root Cause: The original `CMakeLists.txt` attempted to shell out to GNU `make` from within CMake (`add_custom_target(... COMMAND make ...)`), but this approach is fragile: it does not honour CMake's generator, does not propagate build flags (C standard, architecture), and relies on a plain `Makefile` in a sub-directory that does not exist at the expected relative path after the CMake out-of-source build tree is created.

Fix: Replaced the `Makefile-delegation` approach with a native CMake `add_library(softfloat STATIC ...)` that:

- Compiles all `source/*.c` files directly.
- Defines `SOFTFLOAT_FAST_INT64` for the 64-bit fast path.
- Includes the RISC-V specialization directory (`source/RISC-V/`).
- Explicitly excludes M-variant source files that are incompatible with `SOFTFLOAT_FAST_INT64`.
- Sets `C_STANDARD 11` on the target.

Portability: Needed on **all** platforms.

2.2 Hard-coded bare `-lboost_*` linker flags — `platform/sep/CMakeLists.txt`

File: `vp/platform/sep/CMakeLists.txt`

Error:

```
ld: library 'boost_system' not found
```

Root Cause: The `sep-vp` target's `target_link_libraries` call contained the hard-coded raw flags `-lboost_iostreams`, `-lboost_program_options`, and `-lboost_system`. In Boost 1.70+ (and by extension Boost 1.90), `boost_system` is **header-only** — its compiled library was removed entirely. No `libboost_system.{so,dylib,a}` exists in any Boost 1.70+ installation. Additionally, these bare `-l` flags duplicated libraries already provided via the modern `${Boost_LIBRARIES}` CMake imported targets, causing duplicate-library warnings.

Fix: Remove the three bare flags entirely. The required libraries are already covered through their `Boost::` imported targets (found transitively via `${Boost_LIBRARIES}`):

```
# Before (broken):
target_link_libraries(sep-vp
...
    -lboost_iostreams
    -lboost_program_options
    -lboost_system      # does not exist in Boost >= 1.70
)

# After (correct):
target_link_libraries(sep-vp
...
)
```

```
    ${Boost_LIBRARIES} # covers all requested Boost:: targets with full paths
)
```

Portability: This would cause a link failure on **any** platform running Boost 1.70+. Boost 1.70 was released in April 2019; all modern distros (Ubuntu 20.04+, Fedora 32+, etc.) ship with Boost ≥ 1.71 .

3 macOS / Apple Platform-Specific Issues

These defects are **exclusive to macOS** (or macOS ARM64 specifically). Linux builds are unaffected.

3.1 x86 SSE intrinsics included on ARM64 — VeeR-ISS/float.cpp

File: sep/cpu/VeeR-ISS/float.cpp

Error:

```
error: "This header is only meant to be used on x86 and x64 architecture"
#include <emmintrin.h>
```

Root Cause: The VeeR ISS uses x86 SSE2 instructions (`_mm_getcsr`, `_mm_setcsr` — MXCSR register manipulation) to control floating-point rounding mode for SoftFloat compatibility. Apple Silicon Macs use ARM64 and do not have these instructions; `emmintrin.h` guards against inclusion on non-x86 architectures via a `#error`.

Fix: Guard the include and usage with an architecture preprocessor check:

```
#if defined(__x86_64__) || defined(__i386__)
#include <emmintrin.h>
#endif
```

The corresponding MXCSR manipulation code must be similarly guarded, with a fallback no-op or ARM FPCR equivalent on ARM64.

Portability: This is **ARM64 macOS only**. Linux on `x86_64` is unaffected. Linux on AArch64 (e.g., AWS Graviton, Ampere) would be affected by the same issue.

3.2 OVERFLOW / UNDERFLOW macro collision from macOS <math.h> — spi_controller_register.h

File: sep/peripherals/spi_controller/include/spi_controller_register.h

Error:

```
error: expected member name or ';' after declaration specifiers
    unsigned int OVERFLOW : 1;
```

Root Cause: macOS's `<math.h>` (included transitively via SystemC or other headers) defines `OVERFLOW` and `UNDERFLOW` as integer macros (`#define OVERFLOW 3`, `#define UNDERFLOW`

-1). These names collide with bitfield member names in the SPI controller register struct, causing them to be expanded by the preprocessor before the compiler sees the struct definition.

Linux glibc does not define these macros in `<math.h>`, so this is macOS-specific.

Fix: Add `#undef` guards immediately after the problematic headers are included:

```
#ifdef OVERFLOW
#undef OVERFLOW
#endif
#ifdef UNDERFLOW
#undef UNDERFLOW
#endif
```

Portability: macOS-only defect. No change needed for Linux.

3.3 `stdc++fs` library does not exist on macOS — `sep/cpu/CMakeLists.txt`

File: `sep/cpu/CMakeLists.txt`

Error:

ld: library 'stdc++fs' not found

Root Cause: `veeriss_model` unconditionally linked `-lstdc++fs` (the GCC filesystem library). On macOS with Apple Clang, `std::filesystem` has been built into `libc++` since macOS 10.15 (2019) — there is no separate `stdc++fs` library.

On Linux with GCC < 9 , `-lstdc++fs` is needed. On Linux with GCC ≥ 9 , it is also no longer needed. The flag is effectively GCC-only and only relevant for older toolchains.

Fix:

```
target_link_libraries(veeriss_model PUBLIC ... )
if(CMAKE_SYSTEM_NAME STREQUAL "Linux" AND CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
    target_link_libraries(veeriss_model PUBLIC stdc++fs)
endif()
```

Portability: macOS-only failure. On Linux with modern GCC (≥ 9) the conditional evaluates to true but the flag is silently ignored by the linker, so it is safe.

4 SystemC 3.x API Deprecation

4.1 `SC_HAS_PROCESS` deprecated in IEEE 1666-2023

Files: `vp/platform/sep/VeeR-ISSTlm/model/inc/VeeR-ISSTlm.hpp`, `vp/platform/sep/och_sep_ss.hpp`

Warning:

warning: 'sc_has_process_used' is deprecated:
SC_HAS_PROCESS(user_module_name) is obsolete in IEEE 1666-2023
[-Wdeprecated-declarations]

Root Cause: SystemC 3.0.x implements IEEE 1666-2023, which marks SC_HAS_PROCESS() as obsolete. The macro is a no-op in modern SystemC and need not be called; SC_CTOR and modern constructor patterns handle process registration implicitly.

Status: Currently a warning (not an error). Can be silenced project-wide with -DSC_ALLOW_DEPRECATED_IEEE_API, or removed from source for full compliance with IEEE 1666-2023.

Portability: Affects **all** platforms with SystemC 3.0.x.

5 Summary Table of All Changes

#	File	Change Type	Platform	Severity
1	sep/utils/ csml/inc/ csml_report.h	C++20 over- load resolution fix	All	Error
2	vp/ platform/sep/ och_sep_ss.hpp	C++20 con- structor ambi- guity fix	All	Error
3	sep/utils/ csml/inc/csml_register.h	Clang stack- depth pragma	Clang only	Error (warning promoted)
4	sep/ peripherals/ edn/ CMakeLists.txt	Add find_package(OpenSSL) + link	All	Error
5	sep/ peripherals/ sep_memory/ CMakeLists.txt	Add find_package(Boost iostreams) + link	All	Error
6	sep/cpu/VeeR- ISS/softfloat/ CMakeLists.txt	Rewrite to na- tive CMake li- brary	All	Error
7	vp/ platform/sep/ CMakeLists.txt	Remove bare - lboost_* flags	All (Boost ≥ 1.70)	Error
8	vp/platform/ infra/ CMakeLists.txt	Make libvncserver optional; fix SystemC tar- get name	All	Error
9	sep/cpu/VeeR- ISS/float.cpp	Guard emmintrin.h for x86 only	ARM64 (ma- cOS & Linux)	Error
10	sep/ peripherals/spi_controller/ include/spi_controller_register.h	Undefine ma- cOS OVERFLOW/ UNDERFLOW macros	macOS	Error
11	sep/cpu/ CMakeLists.txt	Conditional- ize stdc++fs to Linux/GCC	macOS	Error
12	vp/platform/ run_all_peripherals/ sep/veer/cmake/deprecation/CMakeLists.txt	SC_HAS_PROCESS deprecation after iOS/iOS14 support removed out (double_underscore) Libcov shpper lcoor Sup-	All (SystemC 3.x) macOS iOS Android Linux FreeBSD NetBSD OpenBSD Solaris Windows Hurd DragonFly GnuK Mach	Warning Test Failure (fixed)

6 Runtime & Test-Infrastructure Errors

(run_all_peripherals.sh)

After the sep-vp VP binary was successfully built, sep/peripherals/run_all_peripherals.sh was run to exercise all 17 SEP peripheral models through four test stages: *Debug build*, *ASAN build*, *Coverage build*, and *CTest*. Every single peripheral failed every stage. This section documents all errors encountered, their root causes, and the fixes applied.

6.1 All Debug / ASAN builds failing — C++17 / C++20 ABI mismatch

Symptom (all 17 peripherals):

Undefined symbols for architecture arm64:

```
"sc_core::sc_api_version_3_0_2_cxx201703L::sc_api_version_3_0_2_cxx201703L(...)",  
  referenced from: __cxx_global_var_init in aes_basetest.cpp.o ...  
ld: symbol(s) not found for architecture arm64
```

Root Cause: run_all_peripherals.sh's build_peripheral() function did not pass -DCMAKE_CXX_STANDARD=20 to CMake. Each peripheral's CMakeLists.txt defaults to C++17 when not told otherwise. SystemC 3.0.2 encodes the C++ standard in its ABI sentinel symbol name (cxx201703L = C++17, cxx202002L = C++20). The installed SystemC library was built with C++20, so any translation unit compiled with C++17 references the wrong ABI symbol and the link fails.

Additionally, SYSTEMC_HOME was not being resolved correctly: the auto-discovery section of the script only checked /usr/local/systemc300 and /usr/local/systemc, neither of which exists on the local machine (SystemC is installed at \$HOME/local/systemc-3.0.2-cxx20).

Fix applied to run_all_peripherals.sh: - Expanded the SYSTEMC_HOME candidate list to include \$HOME/local/systemc-3.0.2-cxx20 and ~15 other standard paths used by macOS and common Linux distributions. - Added -DCMAKE_CXX_STANDARD=20 and -DSYSTEMC_HOME=... to both build_peripheral() and run_coverage() cmake invocations.

Portability: This failure would reproduce on any machine where SystemC is compiled with C++20 and the build scripts default to C++17. It is not macOS-specific.

6.2 csrng Debug / ASAN build failing — SystemC::systemc target not found at configure time

Symptom (csrng only):

```
CMake Error at sep/utils/csml/CMakeLists.txt:12 (target_link_libraries):  
  Target "csml_logger" links to: SystemC::systemc  
  but the target was not found.
```

CMake Generate step failed. Build files cannot be regenerated correctly.
fatal error: 'systemc' file not found

Root Cause: sep/peripherals/csrng/CMakeLists.txt adds the CSML subdirectory (add_subdirectory(...csml...)) on line 38, *before* it calls include(FindSystemC) on line 113. When the CSML sub-project runs target_link_libraries(csmlogger PUBLIC SystemC::systemc), the SystemC::systemc imported target does not yet exist — FindSystemC has not been called. CMake warns about the generate step failure but still produces a partial Makefile; the subsequent build then fails because the SystemC include path was never added.

All other peripherals add the CSML subdirectory *after* include(FindSystemC), so they are unaffected.

Fix applied to sep/peripherals/csrng/CMakeLists.txt: Moved the add_subdirectory(csmlogger) call to after include(FindSystemC):

```
# Before (broken — csml added before SystemC is found):
include(FindCCI)
include(FindSystemC) # line 113 — LATER than add_subdirectory

# After (correct):
include(FindCCI)
include(FindSystemC)
if(CMAKE_SOURCE_DIR STREQUAL CMAKE_CURRENT_SOURCE_DIR)
    add_subdirectory ../../utils/csml ${CMAKE_CURRENT_BINARY_DIR}/csml
endif()
```

Portability: This is a source-ordering defect in the csrng CMakeLists.txt that would fail on any platform.

6.3 Coverage build failing — grep: invalid option -- P

Symptom (all 17 peripherals, coverage stage):

```
Coverage      ... grep: invalid option -- P
usage: grep [-abcdDEFGHhIiJLlMmnOopqRSsUVvwXxZz] ...
FAIL
```

Root Cause: The extract_coverage() helper in run_all_peripherals.sh used grep -oP (Perl-compatible regex mode). BSD grep (macOS) does not implement -P; only GNU grep (Linux) supports it. The regex also used \K and (?=) which are Perl-only syntax not available in even -E (extended regex) mode.

Fix applied to run_all_peripherals.sh:

```
# Before (GNU grep only):
grep -oP 'lines\.*:\s*\K[0-9]+\.[0-9]+(?:=)' "$log" | tail -1
```

```
# After (portable – perl is available on all supported platforms):  
perl -ne 'print "$1\n" if /lines\.\+:\s*([0-9]+\.[0-9]+)%/' "$log" | tail -1
```

Portability: macOS-only failure. Linux with GNU grep is unaffected.

6.4 Coverage build failing — FindThreads failure under --coverage flags (macOS)

Symptom (all 17 peripherals, coverage stage, macOS only):

```
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Failed  
-- Looking for pthread_create in pthread - not found  
-- Check if compiler accepts -pthread - no  
CMake Error: Could NOT find Threads (missing: Threads_FOUND)
```

Root Cause: Each peripheral's CMakeLists.txt sets CMAKE_CXX_FLAGS to include --coverage -fprofile-arcs -ftest-coverage *before* calling find_package(Threads REQUIRED). On macOS with AppleClang, the --coverage compile flag alters how the compiler handles test source files used by CMake's FindThreads.cmake probes, causing the -pthread linker test to fail. On Linux the same flag sequence works because GCC's -pthread detection takes a different code path.

Note: on macOS, POSIX threads are part of libSystem and never require a separate library. CMake's probe is unnecessary but mandatory.

Fix applied to run_all_peripherals.sh: Pre-cache the thread detection result for macOS so CMake skips the failing probe:

```
if [[ "$(uname -s)" == "Darwin" ]]; then  
    thread_cache_arg="-DCMAKE_HAVE_LIBC_PTHREAD=1"  
fi
```

Portability: macOS-only workaround. On Linux, thread_cache_arg is empty and has no effect.

6.5 Coverage build failing — ld: library 'gcov' not found (macOS)

Symptom (all 17 peripherals, coverage stage, macOS only):

```
ld: library 'gcov' not found  
clang++: error: linker command failed with exit code 1
```

Root Cause: The peripheral CMakeLists.txt files set:

```
set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} --coverage -lgcov")
```

libgcov is a GCC runtime library. On macOS with AppleClang the equivalent is libclang_rt.profile_osx.a, located in the Clang runtime directory. There is no libgcov installed on macOS.

Fix applied to run_all_peripherals.sh: Create a temporary libgcov.a symlink pointing to libclang_rt.profile_osx.a in a scratch directory, then inject that directory into the linker search path:

```
clang_rt_dir=$(clang -print-runtime-dir)
gcov_shim_dir="${TMPDIR}/gcov-shim-$$"
ln -sf "${clang_rt_dir}/libclang_rt.profile_osx.a" "${gcov_shim_dir}/libgcov.a"
linker_extra_flags="-DCMAKE_EXE_LINKER_FLAGS=-L${gcov_shim_dir}"
```

Portability: macOS-only workaround. On Linux, libgcov is part of GCC and is always available.

6.6 Coverage report generation failing — lcov not installed; LLVM gcov format errors

Symptom 1 (macOS, before install):

```
make[3]: lcov: No such file or directory
```

Symptom 2 (macOS, after brew install lcov):

```
lcov: WARNING: (unsupported) Function begin/end line exclusions not supported
           with this version of GCC/gcov
lcov: ERROR: (inconsistent) ".../tlm_target_socket.h":61: function end line 36
           less than start line 61. Cannot derive function end line.
lcov: ERROR: (format) ".../aes.cpp": unexpected line number '0' for function
           __cxx_global_var_init
```

Root Cause: - lcov is not installed by default on macOS. It must be installed explicitly (brew install lcov). - lcov 2.x introduced stricter validation. When Apple LLVM gcov 17.x is used as the coverage backend (emulating gcov 4.2), the .gcno/.gcda files contain metadata that does not conform to lcov 2.x's expectations: function begin/end line numbers are sometimes inverted for inline and template functions in system headers (tlm_target_socket.h, sc_signal_ifs.h, etc.) and zero line numbers appear for compiler-generated initializers.

Fix applied to run_all_peripherals.sh: Install lcov via Homebrew, then create a wrapper script that injects --ignore-errors flags for all three error categories:

```
cat > "${lcov_wrapper_dir}/lcov" <<'EOF'
#!/usr/bin/env bash
exec "$(which lcov)" --ignore-errors unsupported,unsupported,inconsistent,format
"$@"
EOF
chmod +x "${lcov_wrapper_dir}/lcov"
# Prepend wrapper directory to PATH so cmake's coverage target uses it
export PATH="${lcov_wrapper_dir}:${PATH}"
```

This suppresses the lcov validation errors without modifying any peripheral CMakeLists.txt file.

Portability: macOS-only workaround. On Linux with gcov (GCC), lcov 2.x does not emit these errors.

6.7 Final peripheral test results after all fixes

All 17 peripherals were re-run after the build/infrastructure fixes. The two peripherals that still showed problems — kmac (CTest failure) and edn (no coverage report) — were subsequently root-caused and fixed; see Section 7 and Appendix A. After **all** fixes, **17/17 peripherals pass all four stages**:

Peripheral	Release	ASAN	Coverage	CTest	Coverage %
aes	PASS	PASS	PASS	PASS	92.6%
aon_timer	PASS	PASS	PASS	PASS	98.1%
csrng	PASS	PASS	PASS	PASS	90.8%
edn	PASS	PASS	PASS	PASS	93.1% (was n/a — fixed, §7.2)
efuse	PASS	PASS	PASS	PASS	93.6%
entropy_src	PASS	PASS	PASS	PASS	91.2%
gpio	PASS	PASS	PASS	PASS	87.5%
hmac	PASS	PASS	PASS	PASS	87.1%
key_manager	PASS	PASS	PASS	PASS	88.5%
kmac	PASS	PASS	PASS	PASS	82.7% (was FAIL — fixed, §7.1)
lifecycle_ctrl	PASS	PASS	PASS	PASS	83.3%
mailbox	PASS	PASS	PASS	PASS	97.3%
otbn	PASS	PASS	PASS	PASS	79.3%
secure_dma	PASS	PASS	PASS	PASS	82.1%
spi_controller	PASS	PASS	PASS	PASS	92.6%
spi_flash	PASS	PASS	PASS	PASS	89.2%
uart_16550	PASS	PASS	PASS	PASS	96.1%

7 Functional Defects Root-Caused & Fixed (for Vayavya review)

This section documents the genuine functional/source defects discovered while running the kmac and edn test suites. Unlike the build-system and platform-specific items above, these are **correctness bugs in the model, the test suite, and the source** that are independent of C++20, macOS, or CMake. Every defect below has been root-caused and **fixed**; the exact source diffs are reproduced in **Appendix A** for review.

7.1 kmac — SHAKE/cSHAKE extended-output and cSHAKE reference defects

Peripheral: sep/peripherals/kmac

Original symptom (CTest):

```
TC-031: FAIL - OpenSSL reference failed
TC-032: FAIL - OpenSSL reference failed
TC-034: FAIL - STATE content unchanged after RUN command
TC-044: FAIL - OpenSSL reference failed
TC-094: FAILED - Digest mismatch (prefix validation failed)
TC-182: FAIL - OpenSSL reference failed
[OVERALL RESULT: FAILED - 5 test(s) failed]
```

There were **three distinct root causes**, all reducible to misuse of OpenSSL's extendable-output-function (XOF) API. The key fact: **EVP_DigestFinalXOF()** **finalizes a digest context and may only be called ONCE**. A second call on the same context fails (returns 0) and does not produce further output.

Defect 1 — Test reference helpers call EVP_DigestFinalXOF per block (TC-031, TC-032, TC-044, TC-182)

Files: kmac/test/src/kmac_func002_test.cpp, kmac/test/src/kmac_func003_test.cpp

Classification: Test-suite bug.

The `compute_shake128_reference_blocks`, `compute_shake256_reference_blocks`, and `compute_cshake_reference_blocks` helpers looped over the requested block sizes calling `EVP_DigestFinalXOF` once per block. The second call onward failed, so the helper returned false → the test reported "OpenSSL reference failed". Because a SHAKE/cSHAKE XOF is a *single continuous deterministic byte stream*, squeezing N blocks of sizes `s0`, `s1`, ... is byte-identical to one finalize of length `s0 + s1 + ...`.

Fix: sum the block sizes and call `EVP_DigestFinalXOF` exactly once into the contiguous output buffer. (Appendix A.1)

Defect 2 — Model RUN command calls EVP_DigestFinalXOF a second time (TC-034)

Files: kmac/src/kmac.cpp, kmac/include/kmac.h

Classification: Model bug (functional, affects real register behaviour).

The model's RUN-command (0x31) handler for SHAKE/cSHAKE called `EVP_DigestFinalXOF(ctx, digest_buffer, rate_bytes)` a *second* time to produce the next output block. On OpenSSL this call fails and returns without modifying `digest_buffer` — and crucially **without setting `ERR_CODE`** — so the STATE window was left unchanged. TC-034 (which reads the STATE window before and after RUN and asserts it changes) therefore failed with "STATE content unchanged after RUN command". This same defect would have corrupted the extended output of TC-031/032/044/182 even after Defect 1 was fixed.

Fix: adopt the same buffered-window strategy the KMAC path already uses. During PROCESS, generate the full XOF stream once into `xof_full_output` with a single `EVP_DigestFinalXOF`; each subsequent RUN advances a window through that buffer by `rate_bytes` instead of re-finalizing. The `xof_full_output` buffer was enlarged from 512 → 4096 bytes to hold the 1680 bytes that TC-182 (10 rate-sized blocks) requires. (Appendix A.2)

Defect 3 — TC-094 reference computes plain SHAKE instead of cSHAKE

File: kmac/test/src/kmac_func008_test.cpp

Classification: Test-suite bug. (The model's app-interface cSHAKE128 is correct.)

`compute_cshake128_reference()` computed `SHAKE128("LC_CTRL" || message)`. That is **not** cSHAKE — per NIST SP 800-185, `cSHAKE128(X, L, N, S)` prepends `bytepad(encode_string(N) || encode_string(S), 168)` and applies domain separation. The model's LC_CTRL application interface correctly implements cSHAKE128 with `N=""` and `S="LC_CTRL"`, so the (incorrect) reference never matched, yielding "Digest mismatch (prefix validation failed)". A second, smaller bug: the model absorbs the three full 64-bit beats sent by the test (24 bytes, including the message's trailing zero padding), but the reference hashed only `strlen("Life Cycle Test Message") = 23` bytes.

Fix: rewrite the reference to construct `bytepad(encode_string("") || encode_string("LC_CTRL"), 168)` exactly as the model does, and hash the same 24 bytes the model absorbs. (Appendix A.3)

Result: kmac now passes Release / ASAN / Coverage (82.7%) / CTest — 0 test failures.

7.2 edn — unbalanced LC0V_EXCL_START blocks coverage report

File: sep/peripherals/edn/src/edn.cpp

Classification: Source defect (coverage annotation).

Symptom (coverage stage):

```
lcov: ERROR: (mismatch) ../edn/src/edn.cpp: overlapping exclude directives.  
Found LCOV_EXCL_START at line 1438 - but no matching LCOV_EXCL_STOP  
for LCOV_EXCL_START at line 1350  
make[3]: *** [CMakeFiles/coverage] Error 1
```

Root Cause: `handle_write_ALERT_TEST()` opened a coverage-exclusion region with `// LCOV_EXCL_START` at line 1350 but never closed it with a matching `// LCOV_EXCL_STOP`. The next exclusion region (in `handle_write_ERR_CODE_TEST()` at line 1438) was therefore detected as *overlapping*, which `lcov 2.x` treats as a hard error — aborting the entire coverage build so **no report was produced** (the summary showed `n/a%`).

Fix: add the missing `// LCOV_EXCL_STOP` at the end of the excluded block in `handle_write_ALERT_TEST()`. As defense-in-depth, the `lcov` wrapper in `run_all_peripherals.sh` now also passes `--ignore-errors ...,mismatch` so a future stray marker degrades to a warning rather than killing the report. (Appendix A.4)

Result: `edn` now produces a coverage report at **93.1%**.

8 Recommendations

1. **C++20 overload resolution (`csml_report.h`)** — The `form_report_string` variadic template design is fragile. Consider replacing it with a fold-expression over `std::ostringstream`:

```
template<class... Args>  
inline std::string form_report_string(Args&&... args)  
{  
    std::ostringstream oss;  
    (oss << ... << args);  
    return oss.str();  
}
```

This is a single unambiguous overload, requires no forward declarations, and works correctly in C++17 and C++20.

2. **`csml_reg_vector` deep recursion** — Refactoring `csml_reg_vector<T, N>` to use `std::array<T, N>` would eliminate the 4096-deep template recursion, dramatically improving compile times for the OTBN peripheral and removing the need for the pragma workaround.
3. **SoftFloat build** — The native CMake approach adopted in the fix is the correct long-term solution. The Makefile delegation was inherently fragile and platform-specific.
4. **Cross-platform portability** — All six non-macOS-specific fixes (**items 1–5, 9 above**) indicate issues that would reproduce on any Linux CI environment. It is recommended to add a Linux CI job (e.g., Ubuntu 24.04 + GCC 13 or Clang 17) to catch these earlier.

5. **KMAC xof_full_output capacity** — The buffer is now 4096 bytes (enough for 10 SHAKE128 rate-blocks). If a test ever requests more than 4096 / rate RUN iterations the model will report an error gracefully, but a proper design would use a dynamically-allocated buffer or an EVP_DigestSqueeze-based approach on OpenSSL 3.3+.
-

Appendix A — Source Diffs for Functional Defects (§7)

This appendix contains the full before/after source changes for all four functional defects described in Section 7. All diffs are presented as // BEFORE / // AFTER annotated code blocks so reviewers can apply or validate them without running a diff tool.

A.1 kmac_func002_test.cpp & kmac_func003_test.cpp — XOF reference helpers

Defect: compute_shake128_reference_blocks, compute_shake256_reference_blocks, and compute_cshake_reference_blocks looped EVP_DigestFinalXOF once per block. EVP_DigestFinalXOF may only be called once per context; the second call fails, so the helper returned false → "OpenSSL reference failed".

Files changed: - sep/peripherals/kmac/test/src/kmac_func002_test.cpp (two instances — SHAKE128 and SHAKE256 helpers) - sep/peripherals/kmac/test/src/kmac_func003_test.cpp (one instance — cSHAKE helper)

kmac_func002_test.cpp — compute_shake128_reference_blocks and compute_shake256_reference_blocks

Both functions contained an identical loop body (only the EVP algorithm differed). The change below applies to **both**:

```
// — BEFORE (broken) —————
// Generate output in multiple blocks (successive calls to EVP_DigestFinalXOF)
size_t offset = 0;
for (size_t i = 0; i < num_blocks; i++) {
    if (EVP_DigestFinalXOF(ctx, output + offset, block_sizes[i]) != 1) {
        EVP_MD_CTX_free(ctx);
        return false; // fails on the 2nd call – context already
    }
    offset += block_sizes[i];
}
```

```
// — AFTER (correct) —————
// SHAKE is an XOF: its output is a single continuous, deterministic byte
// stream, so squeezing blocks of sizes s0, s1, ... produces exactly the same
// bytes as one finalize of length (s0 + s1 + ...). EVP_DigestFinalXOF may
// only be called ONCE per context (a second call fails), so we sum the block
// sizes and finalize once into the contiguous output buffer.
```

```

size_t total_len = 0;
for (size_t i = 0; i < num_blocks; i++) {
    total_len += block_sizes[i];
}
if (EVP_DigestFinalXOF(ctx, output, total_len) != 1) {
    EVP_MD_CTX_free(ctx);
    return false;
}

```

kmac_func003_test.cpp — compute_cshake_reference_blocks

```

// — BEFORE (broken) —————
// Generate output in blocks
size_t offset = 0;
for (size_t i = 0; i < num_blocks; i++) {
    if (EVP_DigestFinalXOF(ctx, output + offset, block_sizes[i]) != 1) {
        EVP_MD_CTX_free(ctx);
        return false;
    }
    offset += block_sizes[i];
}

// — AFTER (correct) —————
// cSHAKE is an XOF: its output is a single continuous, deterministic byte
// stream, so squeezing blocks of sizes s0, s1, ... produces exactly the same
// bytes as one finalize of length (s0 + s1 + ...).  EVP_DigestFinalXOF may
// only be called ONCE per context (a second call fails), so we sum the block
// sizes and finalize once into the contiguous output buffer.
size_t total_len = 0;
for (size_t i = 0; i < num_blocks; i++) {
    total_len += block_sizes[i];
}
if (EVP_DigestFinalXOF(ctx, output, total_len) != 1) {
    EVP_MD_CTX_free(ctx);
    return false;
}

```

A.2 kmac.cpp & kmac.h — Model SHAKE/cSHAKE PROCESS and RUN commands

Defect (PROCESS): The SHAKE/cSHAKE branch of `handle_write_CMD_PROCESS()` called `EVP_DigestFinalXOF(ctx, digest_buffer, rate_bytes)` to extract only the first rate-sized block. The EVP context was then fully consumed. Any subsequent RUN command re-called `EVP_DigestFinalXOF` on the dead context, got an error return (without setting `ERR_CODE`), and left `digest_buffer` unchanged.

Defect (RUN): The RUN handler called `EVP_DigestFinalXOF` a second time, which always fails on OpenSSL (context already finalized). The STATE window remained identical to the post-PROCESS state → TC-034 "STATE content unchanged".

Two coordinated changes were required:

A.2a kmac/include/kmac.h — enlarge xof_full_output

```
// — BEFORE —————  
/// @brief Full XOF output buffer for extended output (KMAC/SHAKE RUN commands)  
uint8_t xof_full_output[512];  
  
// — AFTER —————  
/// @brief Full XOF output buffer for extended output (KMAC/SHAKE RUN commands).  
/// Sized to hold multiple rate-sized blocks so successive RUN commands can  
/// window through a single EVP_DigestFinalXOF result (EVP_DigestFinalXOF may  
/// only be called once per context).  
uint8_t xof_full_output[4096];
```

512 bytes was enough for KMAC (single fixed-length output) but TC-182 needs $10 \times 168 = 1680$ bytes of SHAKE128 output. 4096 accommodates 24 SHAKE128 blocks or 30 SHAKE256 blocks before exhaustion.

A.2b kmac/src/kmac.cpp — SHAKE/cSHAKE PROCESS branch

```
// — BEFORE (broken) —————  
// EVP_DigestFinalXOF extracts initial output block (up to rate size)  
// This function can be called multiple times via RUN command for extended output  
EVP_MD_CTX *ctx = static_cast<EVP_MD_CTX *>(evp_md_ctx);  
if (EVP_DigestFinalXOF(ctx, digest_buffer, rate_bytes) != 1) {  
    CSML_ERROR(1, logger)  
        << "OpenSSL EVP_DigestFinalXOF failed for SHAKE/cSHAKE";  
    return false;  
}  
digest_size = rate_bytes;  
CSML_INFO(2, logger) << "SHAKE/cSHAKE initial output finalized: "  
    << digest_size << " bytes (rate)";  
  
// — AFTER (correct) —————  
// EVP_DigestFinalXOF may only be called ONCE per context, and SHAKE/cSHAKE  
// output is a single continuous, deterministic byte stream. Generate a  
// generous buffered output here in one call, expose the first rate-sized  
// block through the STATE window, and let successive RUN commands advance  
// the window through this buffer (matching hardware "squeeze more" semantics  
// where each RUN reveals the next sequential output block).  
EVP_MD_CTX *ctx = static_cast<EVP_MD_CTX *>(evp_md_ctx);  
if (EVP_DigestFinalXOF(ctx, xof_full_output, sizeof(xof_full_output)) != 1) {  
    CSML_ERROR(1, logger)  
        << "OpenSSL EVP_DigestFinalXOF failed for SHAKE/cSHAKE";  
    return false;  
}
```

```

xof_total_length = sizeof(xof_full_output);
xof_output_offset = 0;
std::memcpy(digest_buffer, xof_full_output, rate_bytes);
digest_size = rate_bytes;
CSML_INFO(2, logger) << "SHAKE/cSHAKE initial output finalized: "
                    << digest_size << " bytes (rate), "
                    << xof_total_length << " bytes buffered for RUN";

```

A.2c kmac/src/kmac.cpp — SHAKE/cSHAKE RUN branch (the else block under kmac_en && xof_total_length > 0)

```

// — BEFORE (broken) —————
} else {
    // Non-KMAC XOF (SHAKE/cSHAKE): use EVP_DigestFinalXOF continuation
    EVP_MD_CTX *ctx = static_cast<EVP_MD_CTX *>(evp_md_ctx);
    if (EVP_DigestFinalXOF(ctx, digest_buffer, rate_bytes) != 1) {
        CSML_ERROR(1, logger) << "OpenSSL EVP_DigestFinalXOF failed for RUN";
        return false; // fails silently – ERR_CODE never set, STATE unchanged
    }
    digest_size = rate_bytes;
    CSML_INFO(2, logger) << "RUN: extended output block generated: "
                    << digest_size << " bytes";
}

// — AFTER (correct) —————
} else {
    // Non-KMAC XOF (SHAKE/cSHAKE): advance the STATE window through the
    // buffered XOF output generated during PROCESS. EVP_DigestFinalXOF cannot
    // be called a second time on the same context, so we slice the next
    // rate-sized block out of xof_full_output (each RUN exposes the next
    // sequential output block, matching hardware squeeze semantics).
    xof_output_offset += rate_bytes;

    if (xof_output_offset >= xof_total_length) {
        CSML_ERROR(1, logger)
            << "RUN command: extended output exhausted at offset "
            << xof_output_offset << " (buffered " << xof_total_length << "
bytes)";
        ERR_CODE = 0x08000000 | 0x31; // SwCmdSequence: no more output
        INTR_STATE.kmac_err = 1;
        update_fsm_state(KmacState::ERROR);
        return false;
    }

    size_t bytes_remaining = xof_total_length - xof_output_offset;
    size_t bytes_to_copy =
        std::min(static_cast<size_t>(rate_bytes), bytes_remaining);
    std::memcpy(digest_buffer, xof_full_output + xof_output_offset,
        bytes_to_copy);
    digest_size = static_cast<unsigned int>(bytes_to_copy);

```

```

CSML_INFO(2, logger) << "RUN: advanced SHAKE/cSHAKE STATE window to offset "
<< xof_output_offset << ", showing " << bytes_to_copy
<< " bytes (buffered " << xof_total_length << ")";
}

```

A.3 kmac_func008_test.cpp — TC-094 cSHAKE128 reference and message length

Defect: compute_cshake128_reference() computed SHAKE128(prefix || message) — plain SHAKE, not cSHAKE. Additionally, the call site passed msg_len = strlen(test_msg) = 23 bytes but the model absorbed the full $3 \times 8 = 24$ bytes sent via the app interface.

A.3a The reference function

```

// — BEFORE (plain SHAKE — wrong)
static void compute_cshake128_reference(const char* prefix,
                                       const uint8_t* message,
                                       size_t msg_len, uint8_t* output)
{
    EVP_MD_CTX* ctx = EVP_MD_CTX_new();
    const EVP_MD* md = EVP_shake128();

    EVP_DigestInit_ex(ctx, md, nullptr);
    EVP_DigestUpdate(ctx, prefix, strlen(prefix)); // wrong: no bytepad/encode
    EVP_DigestUpdate(ctx, message, msg_len);
    EVP_DigestFinalXOF(ctx, output, 32);

    EVP_MD_CTX_free(ctx);
}

// — AFTER (proper cSHAKE per NIST SP 800-185)
static void compute_cshake128_reference(const char* prefix,
                                       const uint8_t* message,
                                       size_t msg_len, uint8_t* output)
{
    const size_t rate = 168; // cSHAKE128 rate in bytes

    EVP_MD_CTX* ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_shake128(), nullptr);

    // Build encode_string("") || encode_string(S) per NIST SP 800-185
    size_t custom_len = strlen(prefix);
    uint8_t prefix_block[64];
    size_t prefix_len = 0;
    prefix_block[prefix_len++] = 0x01; // left_encode(0)
    prefix_block[prefix_len++] = 0x00; // encode_string("")
    prefix_block[prefix_len++] = 0x01; // left_encode(len*8)
    prefix_block[prefix_len++] = (uint8_t)(custom_len * 8); // bit-length of S
    std::memcpy(prefix_block + prefix_len, prefix, custom_len);
}

```

```

    prefix_len += custom_len;

    // bytepad(prefix_block, rate) = left_encode(rate) || prefix_block || 0x00...
    uint8_t bytepadded[168];
    size_t offset = 0;
    bytepadded[offset++] = 0x01;
    bytepadded[offset++] = (uint8_t)(rate & 0xFF);
    std::memcpy(bytepadded + offset, prefix_block, prefix_len);
    offset += prefix_len;
    while (offset < rate) bytepadded[offset++] = 0x00;

    EVP_DigestUpdate(ctx, bytepadded, rate);
    EVP_DigestUpdate(ctx, message, msg_len);
    EVP_DigestFinalXOF(ctx, output, 32);

    EVP_MD_CTX_free(ctx);
}

```

A.3b TC-094 call site — message length fix

```

// — BEFORE (wrong message length: 23 bytes, but model absorbs 24) —————
compute_cshake128_reference("LC_CTRL", (const uint8_t*)test_msg, msg_len,
                           reference);

// — AFTER (correct: hash the same 3 × 8 = 24 bytes the model absorbed) ———
// The model absorbs the 3 full 64-bit beats sent above (24 bytes, including
// the trailing zero padding of the message), so the reference must hash the
// same 24 bytes rather than just strlen(test_msg).
compute_cshake128_reference("LC_CTRL", (const uint8_t*)data_words,
                           3 * sizeof(uint64_t), reference);

```

A.4 edn/src/edn.cpp — missing LCOV_EXCL_STOP in handle_write_ALERT_TEST

Defect: LCOV_EXCL_START at line 1350 had no matching LCOV_EXCL_STOP, causing lcov 2.x to abort the entire coverage build with a (mismatch) error.

```

// — BEFORE (missing LCOV_EXCL_STOP – coverage build aborted) —————
// LCOV_EXCL_START - CSML framework W0 register callback execution artifact
if (force_recov || force_fatal)
{
    if (force_recov) { alert_recov_alert.write(true); }
    if (force_fatal) { alert_fatal_alert.write(true); }

    // After delta cycle, restore normal operation via driver
    m_alert_update_event.notify(SC_ZERO_TIME);
}
// ← no LCOV_EXCL_STOP here; next LCOV_EXCL_START in
handle_write_ERR_CODE_TEST
// triggers "overlapping exclude directives" error in lcov 2.x

```

```

    return true;
}

// — AFTER (balanced markers – coverage build succeeds) —————
// LCOV_EXCL_START - CSML framework W0 register callback execution artifact
if (force_recov || force_fatal)
{
    if (force_recov) { alert_recov_alert.write(true); }
    if (force_fatal) { alert_fatal_alert.write(true); }

    // After delta cycle, restore normal operation via driver
    m_alert_update_event.notify(SC_ZERO_TIME);
}
// LCOV_EXCL_STOP ← added

return true;
}

```

Additionally, `run_all_peripherals.sh`'s `lcov` wrapper was updated to tolerate future stray markers:

```

# — BEFORE —————
exec "${real_lcov}" --ignore-errors unsupported,unsupported,inconsistent,format
"$@"

# — AFTER —————
exec "${real_lcov}" --ignore-errors
unsupported,unsupported,inconsistent,format,mismatch "$@"

```

Appendix B — Source Diffs for CSML C++20 Patches (§1.1 and §1.3)

Note for the Vayavya team: The two patches in this appendix affect files inside the `sep/utils/csml` submodule (Vayavya-Labs/CSML), which Tenstorrent cannot modify directly. Both changes are required for C++20 conformance and are tracked for upstream merge. Until they land, CI applies them automatically via a post-checkout Python script embedded in every workflow job that compiles the SEP peripheral library.

B.1 sep/utils/csml/inc/csml_report.h — Replace ambiguous multi-overload set (§1.1)

What changed: The four-template variadic overload set that became ambiguous under C++20's tightened partial-ordering rules was replaced with a single, two-or-more-argument template.

```
// — BEFORE (four templates – ambiguous in C++20) —————

// Forward declaration (string head)
template<class... va_args>
std::string form_report_string(std::string arg1, va_args... args);

// Two-argument generic
template<class T, class U>
inline std::string form_report_string(T arg1, U arg2)
{
    return form_report_string(arg1) + form_report_string(arg2);
}

// Multi-argument generic head
template<class T, class... va_args>
inline std::string form_report_string(T arg1, va_args... args)
{
    return form_report_string(arg1) + form_report_string(args...);
}

// Multi-argument string head
template<class... va_args>
inline std::string form_report_string(std::string arg1, va_args... args)
{
    return arg1 + form_report_string(args...);
}

// — AFTER (single template – unambiguous in C++17 and C++20) —————

template<class First, class Second, class... Rest>
inline std::string form_report_string(First first, Second second, Rest... rest)
{
    return form_report_string(first) + form_report_string(second, rest...);
}
```

Context: The single-argument overloads (`std::string`, `const char*`, and generic `T`) are left unchanged — they serve as the recursion base cases. The replacement collapses the four multi-argument overloads into one that requires at least two parameters, which is always unambiguous regardless of the head type.

B.2 sep/utils/csml/inc/csml_register.h — Clang -Wstack-exhausted pragma guard (§1.3)

What changed: Clang diagnostic push/pop pragmas were added around the csml_reg_vector class template (both the general <T, Quantity> and the base-case <T, 1> specialisation) to suppress the -Wstack-exhausted warning that is promoted to an error by the project's -Werror flag.

// — BEFORE (no pragma – Clang promotes -Wstack-exhausted to an error) —

```
template <class T, unsigned int Quantity>
class csml_reg_vector
{
public:
    csml_reg_vector(std::string name, typename T::memory_type &mem,
                    unsigned int offset, unsigned int spacing)
        : reg_vector(name, mem, offset, spacing), /* ... */ {}
    // ...
    csml_reg_vector<T, Quantity - 1> reg_vector;
};
```

```
template <class T>
class csml_reg_vector<T, 1>
{
public:
    csml_reg_vector(std::string name, typename T::memory_type &mem,
                    unsigned int offset, unsigned int /*spacing*/)
        : reg(name + "_" + std::to_string(0), mem, offset) {}
    // ...
    T reg;
};
```

```
template <class T, unsigned int Quantity1, unsigned int Quantity2>
class csml_reg_2D { /* ... */ };
```

// — AFTER (pragma guards – Clang suppresses the diagnostic; GCC unaffected) –

```
#if defined(__clang__)
#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wstack-exhausted"
#endif
```

```
template <class T, unsigned int Quantity>
class csml_reg_vector
{
public:
    csml_reg_vector(std::string name, typename T::memory_type &mem,
                    unsigned int offset, unsigned int spacing)
        : reg_vector(name, mem, offset, spacing), /* ... */ {}
```

```

    // ...
    csml_reg_vector<T, Quantity - 1> reg_vector;
};

template <class T>
class csml_reg_vector<T, 1>
{
public:
    csml_reg_vector(std::string name, typename T::memory_type &mem,
                    unsigned int offset, unsigned int /*spacing*/)
        : reg(name + "_" + std::to_string(0), mem, offset) {}
    // ...
    T reg;
};

#if defined(__clang__)
#pragma clang diagnostic pop
#endif

template <class T, unsigned int Quantity1, unsigned int Quantity2>
class csml_reg_2D { /* ... */ };

```

Why Clang-only: GCC does not implement `-Wstack-exhausted`. The `#if defined(__clang__)` guards make the pragmas a strict no-op under GCC (including GCC 12 used in the RHEL 8 CI), so the fix is safe for all target platforms.

Long-term recommendation: The `csml_reg_vector<T, N>` recursive template should be refactored to an iterative design (e.g., `std::array<T, N>` with an index-based operator[]) to eliminate the N-deep template instantiation chain and its associated compile-time overhead. This would also remove the need for the Clang pragma workaround.